

Find the Solution for the following:

1. What privilege should a user be given to log on to the Oracle Server? Is this a system or an object privilege?

CREATE SESSION;

2. What privilege should a user be given to create tables?

CREATE TABLE;

3. If you create a table, who can pass along privileges to other users on your table?

GRANT SELECT ON emp TO user1 with grant option;

4. You are the DBA. You are creating many users who require the same system privileges. What should you use to make your job easier?

CREATE role manager role;

GRANT CREATE TABLE, CREATE VIEW TO manager role;

Grant manager-role To user1, user 2;

5. What command do you use to change your password?

ALTER USER username Identified by new password;

6. Grant another user access to your DEPARTMENTS table. Have the user grant you query access to his or her DEPARTMENTS table.

GRANT SELECT ON departments TO user 2;

7. Query all the rows in your DEPARTMENTS table.

Select \* FROM departments;

8. Add a new row to your DEPARTMENTS table. Team 1 should add Education as department number 500. Team 2 should add Human Resources department number 510. Query the other team's table. INSERT into departments Values (500, 'Education');

- INSERT into departments Values (510, 'Human Resources');
9. Query the USER\_TABLES data dictionary to see information about the tables that you own.

SELECT table-name FROM user-tables;

10. Revoke the SELECT privilege on your table from the other team.

Revoke Select on departments FROM USER 2;

11. Remove the row you inserted into the DEPARTMENTS table in step 8 and save the changes.

DELETE from departments WHERE department\_id (500, 510);  
COMMIT;

| <u>Evaluation Procedure</u>    | <u>Marks awarded</u> |
|--------------------------------|----------------------|
| <u>Practice Evaluation (5)</u> | 5                    |
| <u>Viva(5)</u>                 | 5                    |
| <u>Total (10)</u>              | 10                   |
| <u>Faculty Signature</u>       | P.M.                 |



# PROGRAM 1

Write a PL/SQL block to calculate the incentive of an employee whose ID is 110.

DECLARE

v-emp-id employees.employee\_id %TYPE := 110;  
v-salary employees.salary %TYPE;  
v-incentive NUMBER;

BEGIN

SELECT salary INTO v-salary  
FROM employees  
WHERE employee\_id = v-emp-id;

v-incentive := v-salary \* 0.10;

DBMS\_OUTPUT.PUT\_LINE('Employee ID: ' || v-emp-id);

DBMS\_OUTPUT.PUT\_LINE('Salary: ' || v-salary);

DBMS\_OUTPUT.PUT\_LINE('Incentive (10%): ' || v-incentive);

EXCEPTION:

WHEN NO\_DATA\_FOUND THEN

DBMS\_OUTPUT.PUT\_LINE('EMPLOYEE NOT FOUND.');

WHEN OTHERS THEN

DBMS\_OUTPUT.PUT\_LINE('Error: ' || SQLERRM);

END;



## PROGRAM 2

Write a PL/SQL block to show an invalid case-insensitive reference to a quoted and without quoted user-defined identifier.

DECLARE

"My Var" NUMBER := 100;

BEGIN

DBMS\_OUTPUT.PUT\_LINE (My Var);

DBMS\_OUTPUT.PUT\_LINE ("My var");

END;

/



### PROGRAM 3

Write a PL/SQL block to adjust the salary of the employee whose ID 122.

Sample table: employees

DECLARE

v-emp-id employees.employee\_id; TYPE := 122;

BEGIN

UPDATE employees

SET Salary = Salary + (Salary \* 0.10)

WHERE employee-id = v-emp-id;

DBMS\_OUTPUT.PUT\_LINE('Salary updated successfully

for employee ID: ' || v-emp-id);' EXCEPTION

WHEN NO\_DATA\_FOUND THEN

DBMS\_OUTPUT.PUT\_LINE('Employee not found.');

WHEN OTHERS THEN

DBMS\_OUTPUT.PUT\_LINE('Error: ' || SQLERRM);

END



#### PROGRAM 4

Write a PL/SQL block to create a procedure using the "IS [NOT] NULL Operator" and show AND operator returns TRUE if and only if both operands are TRUE.

CREATE OR REPLACE PROCEDURE check\_nulls IS

a Number := 10;

b NUMBER := NULL;

BEGIN

IF a IS NOT NULL AND b IS NOT NULL THEN

DBMS\_OUTPUT.PUT\_LINE ('Both Values are NOT NULL');

ELSE

DBMS\_OUTPUT.PUT\_LINE ('At least one Value is NULL');

END IF;

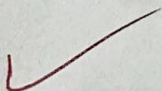
END;

/

BEGIN:

check\_nulls;

END;





# PROGRAM 5

Write a PL/SQL block to describe the usage of LIKE operator including wildcard characters and escape character.

DECLARE

V\_name VARCHAR2 (20) := 'RASESH';

BEGIN

IF V\_name LIKE 'RAY.' THEN

DBMS\_OUTPUT.PUT\_LINE ('Name Starts with R');

END IF;

IF V\_name LIKE 'R\_JESH' THEN

DBMS\_OUTPUT.PUT\_LINE ('Second Character is any Single letter');

END IF;

IF 'A#B' LIKE 'A#\_Y.' ESCAPE '#' THEN

DBMS\_OUTPUT.PUT\_LINE ('Escape character used correctly');

END IF;

END;



# PROGRAM 6

Write a PL/SQL program to arrange the number of two variable in such a way that the small number will store in num\_small variable and large number will store in num\_large variable.

DECLARE

a NUMBER := 50;

b NUMBER := 30;

num\_small NUMBER;

num\_large NUMBER;

BEGIN

IF a < b THEN

num\_small := a;

num\_large := b;

ELSE

num\_small := b;

num\_large := a;

END IF;

DBMS\_OUTPUT.PUT\_LINE ('Small Number: ' || num\_small);

DBMS\_OUTPUT.PUT\_LINE ('Large Number: ' || num\_large);

END;



## PROGRAM 7

Write a PL/SQL procedure to calculate the incentive on a target achieved and display the message either the record updated or not.

BEGIN

UPDATE employees

SET Salary = salary + (Salary \* 0.05)

WHERE employee\_id = 110 AND sales >= target;

IF SQL % Row COUNT > 0 THEN

DBMS\_OUTPUT.PUT\_LINE ('Incentive added.');

ELSE

DBMS\_OUTPUT.PUT\_LINE ('Record not updated.');

ENDIF;

END;



### PROGRAM 8

Write a PL/SQL procedure to calculate incentive achieved according to the specific sale limit.

CREATE OR REPLACE PROCEDURE calc\_incentive IS

Sales NUMBER := 80000;

Incentive NUMBER;

BEGIN

IF Sales  $\geq$  100000 THEN

incentive := Sales \* 0.10;

ELSEIF Sales  $\geq$  50000 then

incentive := Sales \* 0.05

ELSE

incentive := 0;

END IF;

DBMS\_OUTPUT.PUTLINE ('Sales: ' || sales);

DBMS\_OUTPUT.PUTLINE ('Incentive: ' || incentive);

END;

BEGIN

calc\_incentive;

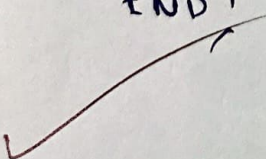
END;



### PROGRAM 9

Write a PL/SQL program to count number of employees in department 50 and check whether this department have any vacancies or not. There are 45 vacancies in this department.

```
DECLARE
    v-count NUMBER;
    v-Vacancies NUMBER := 45;
BEGIN
    SELECT COUNT (*) INTO v-count
    FROM employees
    WHERE department_id = 50;
    IF v-count < v-Vacancies THEN
        DBMS-OUTPUT.PUT-LINE('Vacancies available: ' || (v-Vacancies - v-count));
    ELSE
        DBMS-OUTPUT.PUT-LINE('No vacancies in department 50');
    END IF;
END;
```





# PROGRAM 10

Write a PL/SQL program to count number of employees in a specific department and check whether this department have any vacancies or not. If any vacancies, how many vacancies are in that department.

```
DECLARE
```

```
  v_dept_id NUMBER := 60;
```

```
  v_total_positions NUMBER := 50;
```

```
  v_emp_count NUMBER;
```

```
BEGIN
```

```
  SELECT COUNT(*) INTO v_emp_count
```

```
  FROM employees
```

```
  WHERE department_id = v_dept_id;
```

```
IF v_emp_count < v_total_positions THEN
```

```
  DBMS_OUTPUT.PUT_LINE ('Vacancies available?'
```

```
    || (v_total_positions - v_emp_count))
```

```
ELSE
```

```
  DBMS_OUTPUT.PUT_LINE ('No vacancies in Department'
```

```
    || v_dept_id);
```

```
END IF;
```

```
END;
```

```
/
```



# PROGRAM 11

Write a PL/SQL program to display the employee IDs, names, job titles, hire dates, and salaries of all employees.

```
DECLARE
    CURSOR emp_cur IS
        SELECT employee_id, first_name, job_id, hire_date, salary
        FROM employees;
    v_emp emp_cur%ROWTYPE;
BEGIN
    OPEN emp_cur;
    LOOP
        FETCH emp_cur INTO v_emp;
        EXIT WHEN emp_cur%NOT FOUND;
        DBMS_OUTPUT.PUTLINE ('ID: ' || v_emp.employee_id ||
                               ', Name: ' || v_emp.first_name ||
                               ' Job: ' || v_emp.job_id ||
                               ' Hire Date: ' || v_emp.hire_date ||
                               ' Salary: ' || v_emp.salary);
    END LOOP;
    CLOSE emp_cur;
END;
```



## PROGRAM 12

Write a PL/SQL program to display the employee IDs, names, and department names of all employees.

DECLARE

CURSOR emp-cur IS

SELECT e.employee-id, e.first-name, d.department-name

FROM employees e

JOIN department d

ON e.department-id = d.department-id;

V-rec emp-cur %ROWTYPE;

BEGIN

OPEN emp-cur;

LOOP

FETCH emp-cur INTO V-rec;

EXIT WHEN emp-cur %NOTFOUND;

DBMS-OUTPUT-PUTLINE ('ID: ' || V-rec.employee-id ||

' Name: ' || V-rec.first-name ||

' Department: ' || V-rec.department-name);

END LOOP;

CLOSE emp-cur;

END;

/



### PROGRAM 13

Write a PL/SQL program to display the job IDs, titles, and minimum salaries of all jobs.

```
DECLARE
  CURSOR job_cur IS
    SELECT job-id, job-title, min-salary
      FROM jobs;
  v_job job_cur%ROWTYPE;
BEGIN
  OPEN job_cur;
  LOOP
    FETCH job_cur INTO v_job;
    EXIT WHEN job_cur%NOT FOUND;
    DBMS_OUTPUT.PUT_LINE('Job ID: ' || v_job.job-id,
                          'Title: ' || v_job.job-title ||
                          'MinSalary: ' || v_job.min-salary);
  END LOOP;
  CLOSE job_cur;
```



# PROGRAM 14

Write a PL/SQL program to display the employee IDs, names, and job history start dates of all employees.

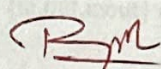
```
DECLARE
    CURSOR emp_cur IS
        SELECT e.employee-id, e.first-name, jh.start-date
        FROM employees e
        JOIN job-history jh
        ON e.employee-id = jh.employee-id;
    v_rec emp_cur%ROWTYPE;
BEGIN
    OPEN emp_cur;
    LOOP
        FETCH emp_cur INTO v_rec;
        EXIT WHEN emp_cur%NOTFOUND;
        DBMS_OUTPUT.PUT_LINE('ID: ' || v_rec.employee-id ||
                               'Name: ' || v_rec.first-name ||
                               ', Start Date: ' || v_rec.start-date);
    END LOOP;
    CLOSE emp_cur;
END;
```



# PROGRAM 15

Write a PL/SQL program to display the employee IDs, names, and job history end dates of all employees.

```
BEGIN
  FOR rec IN (
    SELECT e.employee-id, e.first-name, jh.end-date
    FROM employees e JOIN job-history jh
    ON e.employee-id = jh.employee-id
  ) LOOP
    DBMS_OUTPUT.PUT_LINE('ID: ' || rec.employee-id ||
      ' Name: ' || rec.first-name ||
      ' End Date: ' || rec.end-date);
  END LOOP;
END;
```

| Evaluation Procedure  | Marks awarded                                                                       |
|-----------------------|-------------------------------------------------------------------------------------|
| PL/SQL Procedure(5)   | 5                                                                                   |
| Program/Execution (5) | 5                                                                                   |
| Viva(5)               | 5                                                                                   |
| Total (15)            | 15                                                                                  |
| Faculty Signature     |  |